CSL7520 GPU Programming Assignment Report
Problem 5-GPU-Based K-Means Clustering Using CUDA

25th October 2026

Submitted by: Group Sr.No.24

- Sanjay Marathe (PGDAI, Roll no. G25AIT2096)
- Mahadev Vishwakarma (PGDAI, Roll no. G25AIT2131)
- Rohit Raghuram (PGDAI, Roll no. G25AIT2145)

==========

1. Introduction

In this assignment we wrote K-means clustering in CUDA. We also wrote a normal CPU program with one thread, so that we can compare the GPU with it. K-means puts N points, each with D features, into K clusters. In every iteration we find the squared distance of each point to each centroid:

$$d(x_i, c_k) = \Sigma(j=1..D)\ (x_i,j - c_k,j)^2$$

Then each point goes to its nearest centroid, and each centroid moves to the mean of its points. We stop when no point changes its cluster, when the centroids move less than the tolerance, or when the maximum number of iterations is reached. The task is to do the distance and the centroid update in parallel, to handle many threads adding to the same centroid (we tried atomics and reduction), to use shared memory, and to compare time and results for different N, D and K.

2. Data and Setup

We used the Instagram file- Instagram visits clustering.csv (2,600 rows: user ID, visit score and spending rank). The user ID is dropped, so D = 2, and we cluster all rows with K = 5, using the default 300 iterations and tolerance 0.0001. This data is too small to show speed differences, so for the speed tests we also made random data. We pick K random centres between 0 and 100 and make each point a random centre plus noise (standard deviation 5). We used N = 10,000, 100,000 and 1,000,000, D = 2, 8 and 32, and K = 4, 16 and 64, with a maximum of 50 iterations. The case N = 1,000,000, D = 32, K = 64 was skipped because the CPU would take too long, so 26 cases were run.

The CPU and all GPU variants start from the same centroids (K data points picked with seed 42). Everything ran on Google Colab with a Tesla T4 GPU and CUDA 13.0. The CPU code was compiled with g++ -O3 and the CUDA code with nvcc -O3 for sm_75.

3. Method

The program takes N, D, K, the data points, the starting centroids, the maximum number of iterations and the tolerance as input. The points come from the CSV file, or for random data

we give N and D in the command. The starting centroids can be given in a file, otherwise K random points are used. The output is the cluster of every point, the final centroids, the number of iterations and the time. The clusters and the centroids are also saved in CSV files.

The CPU program uses normal loops. It checks every point against every centroid and then finds the new centroids by taking the average of the points in each cluster.

In the GPU program, one thread handles one point. The data is copied to the GPU once and stays there. Each iteration has three steps, and each step is one kernel:

1. The first kernel finds the nearest centroid for each point.

2. The second kernel adds up the points of each cluster. Many threads write to the same cluster here, so this was the hard part (see Section 4).

3. The third kernel divides the sums by the counts to get the new centroids.

After this, the CPU checks if it should stop.

For timing, we measured only the loop on both CPU and GPU. For the GPU we ran each variant 3 times and took the best time. Copying the data to the GPU was timed separately. To check that the GPU gives the same answer as the CPU, we compared their final inertia. Inertia is the sum of squared distances from the points to their centroids.

4. GPU Variants

We wrote seven GPU variants (Table 1). Most of them differ by one change, so we can see what makes the program faster.

Table 1. The seven GPU variants

No.	Points stored	Centroids in assign step	Update step
1	row by row (Array of Structures- AoS)	global memory	global atomic adds
2	row by row (AoS)	shared memory	global atomic adds
3	row by row (AoS)	shared memory	block table in shared memory
4	column by column (Structure of Arrays- SoA)	shared memory	global atomic adds
5	column by column (SoA)	shared memory	block table in shared memory
6	column by column (SoA)	shared memory	warp-shuffle tree reduction
7	column by column (SoA)	shared memory	block table with padded rows

AoS means the numbers of one point are stored together. SoA means the same feature of all points is stored together.

In variants 1, 2 and 4, every thread adds its point directly into global memory. Many threads write to the same place. These variants were slower than the block table in most cases.

In variants 3, 5 and 7, each block adds its points into a small table first. Then it adds the table to global memory once. This was much faster.

Variant 6 adds the values of the threads together in steps. It was mostly the fastest when K = 4.

Variant 7 is the same as variant 5, but with a slightly bigger table. Variant 5 was slow when D = 32 and K was 16 or 64, and variant 7 fixed this.

Warp divergence

Threads in a warp can slow down each other if they take different paths in the code. We changed the assign kernel so that all threads follow the same path. The profiler showed no split paths in this kernel. We did not change the other kernels.

5. Correctness

For the Instagram data the CPU stopped after 8 iterations with inertia 2.784075e+05. Table 2 shows the centroids and Figure 1 the clusters. In all seven GPU variants 0 of the 2,600 points had a different cluster than on the CPU. The three stopping rules (maximum iterations, tolerance, stable assignments) and our own starting centroids also gave the same results on the CPU and the GPU.

Table 2. Final centroids for the Instagram data (K = 5)

Cluster	Visit score	Spending rank	Points
0	39.27	65.26	398
1	89.26	19.96	791
2	23.95	24.97	500
3	60.01	19.74	311
4	79.62	85.02	600

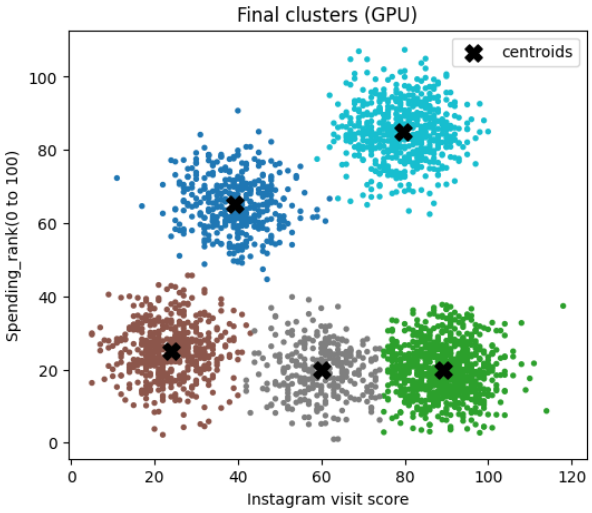


Figure 1. Final clusters of the Instagram data

For the random data the inertia of every GPU variant was within 0.004% of the CPU inertia in all 26 cases. The number of iterations was the same in 23 cases, and in the other 3 some variants stopped 1 or 2 iterations earlier or later. In some cases that hit the 50 iteration limit the centroids differed by up to 0.64, but the inertia was almost the same.

6. Results

6.1 Speed-up over the CPU

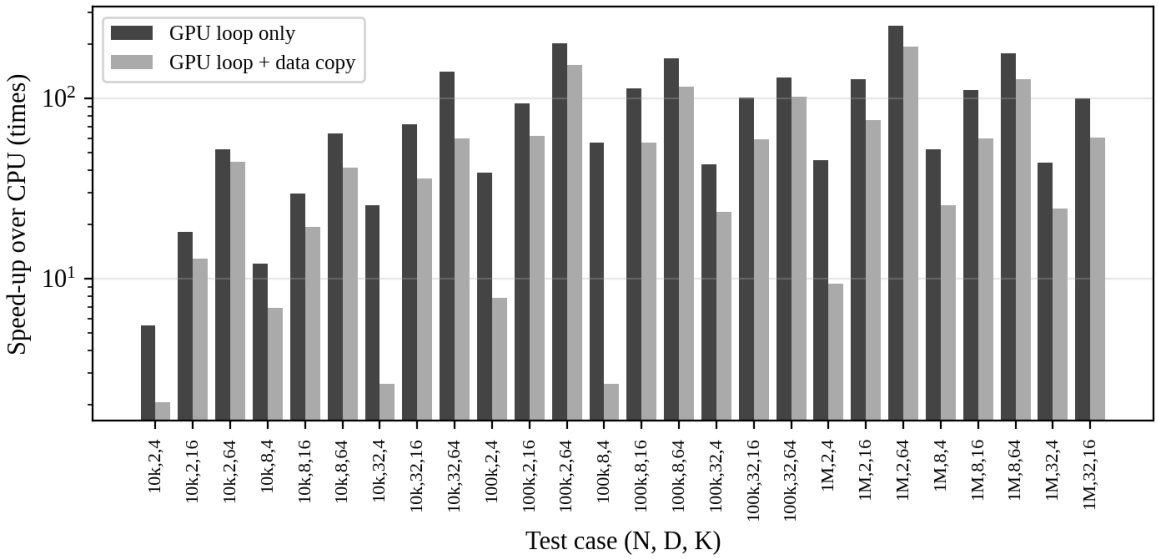


Figure 2. Speed-up of the fastest GPU variant over the CPU (log scale)

The best GPU variant was faster than the CPU in all 26 cases. The loop alone was 5.5 to 254.8 times faster, and at least 39 times faster when N was 100,000 or more. The biggest speed-up was for N = 1,000,000, D = 2, K = 64 (8,409 ms on the CPU and 33.0 ms on the GPU). When the data copy is included, the speed-up is 2.1 to 194.7 times. On the small Instagram data the

GPU was only about as fast as the CPU (0.35 ms on the CPU and 0.22 to 0.33 ms on the GPU). The full table is in Appendix A.

6.2 Which variant is better?

The block table gave the biggest speed-up. It was faster than global atomics in most cases, up to about 8 times. It was slower only when D = 32 and K was 16 or 64. The padded version (variant 7) fixed this problem. SoA was faster than AoS when D was large, but for D = 2 there was little difference. Keeping the centroids in shared memory made almost no difference. The tree reduction was mostly best when K = 4.

Table 3. Fastest variant for different K and D (26 cases)

Case	Best variant	Cases won
K = 4	6 (tree reduction)	7 of 9 (variant 7 in the other 2)
K = 16 or 64, D = 2	3 or 5 (block table)	all 6
K = 16 or 64, D = 8 or 32	7 (padded block table)	all 11

7. Conclusion and Limitations

The GPU gave the same results as the CPU, and it was much faster. The best GPU version was 5.5 to 254.8 times faster than the CPU with one thread. The biggest gains came from adding the points in a small table in shared memory first, storing the data feature by feature (SoA), and making the table a bit wider (variant 7). Putting the centroids in shared memory did not help. The tree reduction was only best when K = 4.

Our results have some limits. The CPU program used only one thread, and we did not compare with a faster CPU. The timings changed a little each time we ran the code. Many of the cases stopped at 50 iterations before they finished, and we did not check how good the clusters are. We tested only one GPU (Tesla T4) and only a few values of N, D and K.

Appendix A. Full results

Fastest of the GPU variants 1 to 7 in each case against one CPU thread. Times are in milliseconds. The last column includes the data copy to the GPU.

N	D	K	CPU (ms)	Best GPU (ms)	Best variant	Speed-up	Speed-up with copy
10,000	2	4	0.919	0.167	7	5.5×	2.1×
10,000	2	16	12.22	0.671	5	18.2×	13.0×
10,000	2	64	86.25	1.654	5	52.1×	44.6×
10,000	8	4	9.217	0.760	6	12.1×	6.9×
10,000	8	16	36.00	1.208	7	29.8×	19.4×
10,000	8	64	67.01	1.049	7	63.9×	41.2×

N	D	K	CPU (ms)	Best GPU (ms)	Best variant	Speed-up	Speed-up with copy
10,000	32	4	5.247	0.204	6	25.7×	2.6×
10,000	32	16	135.3	1.873	7	72.3×	36.1×
10,000	32	64	194.8	1.383	7	140.9×	60.0×
100,000	2	4	13.86	0.355	6	39.0×	7.8×
100,000	2	16	231.2	2.449	3	94.4×	61.9×
100,000	2	64	848.6	4.165	3	203.7×	154.2×
100,000	8	4	11.77	0.207	6	56.9×	2.6×
100,000	8	16	477.0	4.167	7	114.5×	57.3×
100,000	8	64	1,596	9.508	7	167.8×	116.7×
100,000	32	4	844.0	19.53	6	43.2×	23.6×
100,000	32	16	2,391	23.57	7	101.4×	59.6×
100,000	32	64	8,079	61.85	7	130.6×	102.9×
1,000,000	2	4	124.9	2.738	6	45.6×	9.4×
1,000,000	2	16	2,179	16.94	3	128.6×	76.3×
1,000,000	2	64	8,409	33.00	3	254.8×	194.7×
1,000,000	8	4	1,908	36.59	6	52.2×	25.8×
1,000,000	8	16	4,799	42.92	7	111.8×	60.3×
1,000,000	8	64	16,270	90.75	7	179.3×	127.6×
1,000,000	32	4	8,517	192.1	7	44.3×	24.6×
1,000,000	32	16	23,520	234.3	7	100.4×	61.1×